

Android Development

Excuse Me

Projekt Dokumentáció

Tamasi Krisztian Erik
Hallgatói azonosító: 115

2026. március 23.

Tartalomjegyzék

1	Bevezetés	3
1.1	Motiváció	3
1.2	Célkitűzések	3
1.3	Létező megoldások és piaci körkép	3
2	Felhasznált technológiák	4
2.1	Frontend technológiák	4
2.2	Backend technológiák	4
2.3	Adatkezelés	4
2.4	Fejlesztői környezet és eszközök	4
3	Rendszertervezés	5
3.1	Funkcionális és nem-funkcionális követelmények	5
3.2	Rendszerarchitektúra	5
3.3	Mobilalkalmazás tervezése	6
3.4	Szerver és API tervezése	6
3.5	Adatbázis-modell	6
4	Implementáció: Mobilalkalmazás	7
4.1	Felhasználói felület megvalósítása	7
4.2	Állapotkezelés és üzleti logika	7
4.3	Külső szolgáltatások integrációja	7
5	Implementáció: Szerveroldal	7
5.1	Szerverstruktúra és modulok	7
5.2	Hitelesítés és jogosultságkezelés	8
5.3	Adatbázis-műveletek és optimalizálás	8
6	Kommunikáció és Integráció	8
6.1	Protokollok	8
6.2	Adatcsere formátumok	8
6.3	Összehangolás	8

7	Tesztelés és Eredmények	9
7.1	Egységtesztek és integrációs tesztek	9
7.2	Felhasználói tesztelés visszajelzései	9
7.3	Teljesítménymérés	9
8	Következtetések és Jövőbeli tervek	10
8.1	Elért eredmények összefoglalása	10
8.2	Ismert korlátok és hibák	10
8.3	Tovább lépési lehetőségek	10
9	Mellékletek és Referenciák	10
9.1	Irodalomjegyzék	10
9.2	Forráskód-részletek	10

1 Bevezetés

1.1 Motiváció

Az **Excuse Me** projekt ötlete abból a hétköznapi helyzetből indult ki, hogy az emberek gyakran próbálnak gyorsan elfogadható, vicces vagy éppen komoly hangvételű magyarázatot adni kellemetlen szituációkra. Ilyen lehet a késés, egy elmulasztott találkozó, egy félresikerült családi program vagy egy kínos munkahelyi helyzet. A projekt célja egy olyan mobilalkalmazás létrehozása volt, amely egyetlen valós bemenetből stílusos, megosztható és szórakoztató alibit készít.

A probléma technikai oldalról is érdekes: a felhasználói élmény akkor jó, ha az alkalmazás gyors, mobilon jól használható, a szerveroldali AI-hívások biztonságosak, a megosztott tartalmak pedig valamilyen közös felületen is visszakereshetők. Emiatt a projekt nem csupán egy egyszerű UI-gyakorlat, hanem egy teljes kliens–szerver rendszer.

1.2 Célkitűzések

A projekt minimum viable product (MVP) szintű céljai a következők voltak:

- mobilalkalmazás készítése Flutterrel;
- felhasználói szöveg bekérése és két hangnem támogatása: **goofy** és **serious**;
- backend szolgáltatás készítése FastAPI-val;
- AI-alapú szöveggenerálás szerveroldali integrációval;
- felhasználói azonosítás, előzmények és alap statisztikák kezelése;
- nyilvános “Wall of Shame” nézet megvalósítása reakciókkal és admin moderációval.

A végső elvárás tehát nem csak egy működő generátor volt, hanem egy olyan demonstrálható rendszer, amely bemutatja a mobil frontend, a REST API, a felhőalapú adatkezelés és az AI-szolgáltatás integrációját is.

1.3 Létező megoldások és piaci körkép

Hasonló funkciók részben megtalálhatók általános AI-chat alkalmazásokban, szövegíró asszisztensekben, illetve egyszerű “excuse generator” weboldalakon. Ezek azonban tipikusan valamelyik alábbi korláttal rendelkeznek:

- nem mobil-first élményt adnak;
- nem tárolják strukturáltan a korábbi generálásokat;
- nem támogatják a közösségi megosztást és reakciókat;
- nem biztosítanak dedikált kategorizálást, statisztikát vagy moderációt.

Az **Excuse Me** ettől abban különbözik, hogy a generálást egy célzott, humoros use case köré építi, miközben teljes értékű mobilalkalmazásként működik. A rendszerben megtalálható a hitelesítés, az előzménykezelés, a kategóriaalapú rendezés, a toplista, valamint az admin jogosultságon alapuló tartalomkezelés is.

2 Felhasznált technológiák

2.1 Frontend technológiák

A mobil kliens **Flutter** alapokra épül, Dart nyelven. A választás mellett szólt, hogy gyors fejlesztést tesz lehetővé, egy kódbázisból több platform célozható, valamint jól támogatja a modern, egyedi megjelenésű felhasználói felületek kialakítását [3].

A projekt frontend rétege tartalmaz:

- többképernyős navigációt;
- saját vizuális témákat;
- HTTP-alapú kommunikációt a backenddel;
- Firebase inicializálást és Firestore-integrációt;
- lokális session-visszaállítást.

2.2 Backend technológiák

A szerveroldal **Python** nyelven, **FastAPI** keretrendszerrel készült [2]. Ez a technológia jól illeszkedik JSON API-k gyors elkészítéséhez, támogatja a típusos adatmodelleket, az aszinkron működést és az automatikus validációt.

A backend fő feladatai:

- hitelesítés és tokenkezelés;
- generálási kérelmek fogadása;
- OpenRouter alapú AI-hívás;
- előzmények és nyilvános bejegyzések mentése;
- statisztikák és toplisták kiszolgálása.

2.3 Adatkezelés

Az adatkezelés **Firestore** használatával valósul meg [1]. A rendszer fő kollekciói a felhasználók, a generált alibik és a nyilvános fal bejegyzései. A fejlesztéshez és a teszteléshez emellett rendelkezésre áll egy **InMemoryRepository** is, amely gyors izolált tesztekhez tesz lehetővé adatbázis nélkül.

2.4 Fejlesztői környezet és eszközök

A projekt monorepo formában szervezett Git tárhelyen található. A főbb eszközök:

- **Git** verziókezelés;
- **Android Studio** és **VS Code** fejlesztői környezet;
- **pytest** backend tesztekhez;
- **flutter_test** widget tesztekhez;
- **Render** backend telepítéshez [4].

3 Rendszertervezés

3.1 Funkcionális és nem-funkcionális követelmények

Funkcionális követelmények:

- regisztráció és bejelentkezés;
- alibi generálás két stílusban;
- generálások mentése előzmények közé;
- publikálás a nyilvános falra;
- reakciók kezelése;
- kategória szerinti szűrés és toplisták;
- admin alapú törlés.

Nem-funkcionális követelmények:

- rövid válaszidő a mobil felületen;
- áttekinthető és könnyen használható UI;
- szerveroldali kulcskezelés, hogy az API-kulcs ne kerüljön kliensoldalra;
- tesztelhető, moduláris architektúra;
- bővíthető adattárolási réteg.

3.2 Rendszerarchitektúra

A rendszer magas szintű architektúrája kliens–szerver modellre épül. A mobilalkalmazás HTTP kéréseket küld a FastAPI backendnek, amely egyrészt AI generálást kér az OpenRouter szolgáltatástól, másrészt Firestore-ban tárolja a tartalmakat.

Flutter mobil kliens → FastAPI backend → OpenRouter

Flutter mobil kliens ↔ Firestore

A fenti egyszerűsített ábra a fő adatáramlást szemlélteti. Ha később külön diagram készül, ide behelyezhető a docs/architecture-diagram.png állomány.

1. ábra: A rendszer magas szintű architektúrája

3.3 Mobilalkalmazás tervezése

A mobilalkalmazás fő szempontja az volt, hogy a leggyakoribb művelet, vagyis az alibi generálás minél kevesebb lépésből elérhető legyen. A kezdőképernyőn a felhasználó rögtön megadhatja a valódi okot, kiválaszthatja a kívánt stílust, majd elindíthatja a generálást.

A navigációs struktúra egyszerű, de bővíthető:

- **Generator** fül;
- **Wall of Shame** fül;
- oldalsó drawer a statisztikákhoz, előzményekhez, kategóriákhoz és toplistához.

Az UI/UX alapelvek közé tartozott az erős vizuális identitás, a gyors visszajelzés, valamint az, hogy a fő funkciók már vendég módban is láthatók legyenek, miközben bizonyos műveletek hitelesítéshez kötöttek.

3.4 Szerver és API tervezése

A szerver REST szemléletű API-t biztosít. A legfontosabb végpontok:

Végpont	Módszer	Funkció
/health	GET	Egészségügyi ellenőrzés
/api/auth/signup	POST	Regisztráció
/api/auth/login	POST	Bejelentkezés
/api/excuses/generate	POST	Alibi generálása
/api/wall/publish/{generation_id}	POST	Generálás publikálása
/api/wall	GET	Nyilvános fal lekérdezése
/api/wall/react/{post_id}	POST	Reakció hozzáadása
/api/history	GET	Felhasználói előzmények
/api/stats/overview	GET	Felhasználói statisztikák
/api/categories/{category}	GET	Kategória szerinti lista
/api/leaderboard	GET	Toplista
/api/admin/wall/{post_id}	DELETE	Admin törlés

3.5 Adatbázis-modell

A projekt dokumentum-orientált adattárolást alkalmaz. A legfontosabb logikai entitások:

- **users:** felhasználónév, jelszó hash, admin státusz, létrehozási idő;
- **excuse_generations:** user azonosító, eredeti igazság, generált szöveg, stílus, kategória, nyelv, publikálási állapot;
- **wall_posts:** nyilvános bejegyzések, reakciók, kapcsolódó generálás azonosítója.

A kapcsolat logikailag egy-a-több: egy felhasználónak több generálása lehet, és egy generálásból legfeljebb egy nyilvános bejegyzés készül.

4 Implementáció: Mobilalkalmazás

4.1 Felhasználói felület megvalósítása

A mobil felület központi eleme egy `StatefulWidget` alapú kezdőképernyő, amely kezeli a fűles nézetet, az autentikációs állapotot és a generálási folyamat vizuális visszajelzéseit. A kezelőfelület több egyedi widgetből áll, például stílusválasztóból, eredménykártyából és témaváltóból.

4.2 Állapotkezelés és üzleti logika

Az alkalmazás a projekt jelenlegi méretére szabott, lokális állapotkezelést használ. A felhasználói session, a kiválasztott stílus, a generált válasz, valamint a hibák és töltési állapotok közvetlenül a fő képernyő állapotában kerülnek tárolásra. Ez egyszerű, jól követhető megoldás MVP szinten.

4.3 Külső szolgáltatások integrációja

A mobil kliens két külső irányba kommunikál:

- a backend REST API felé HTTP-n keresztül;
- a Firebase/Firestore felé a nyilvános tartalmak és megfigyelt listák kezeléséhez.

Az alábbi kódrészlet szemlélteti, hogy a fő alkalmazás hogyan indul és hogyan inicializálja a Firebase környezetet:

Listing 1: Flutter alkalmazásindítás és Firebase inicializálás

```
1 Future<void> main() async {  
2   WidgetsFlutterBinding.ensureInitialized();  
3   try {  
4     await Firebase.initializeApp(  
5       options: DefaultFirebaseOptions.currentPlatform,  
6     );  
7   } catch (_) {  
8     // The app shell can still run without configured Firebase  
9     // credentials.  
10  }  
11  runApp(const ExcuseMeApp());  
12 }
```

5 Implementáció: Szerveroldal

5.1 Szerverstruktúra és modulok

A backend több kisebb modulra bontott:

- `main.py`: végpontok és dependency injection;
- `auth.py`: jelszó hash és JWT-szerű tokenkezelés;

- `openrouter.py`: AI szolgáltatás integráció;
- `repository.py`: adattárolási absztrakció és Firestore implementáció;
- `models.py`: kérés- és válaszmodellek Pydantic segítségével.

5.2 Hitelesítés és jogosultságkezelés

A rendszer token alapú hitelesítést használ. Regisztráció és bejelentkezés után a backend egy aláírt token ad vissza, amelyet a kliens a későbbi védett kérésekhez használ. A jelszavak nem sima szöveggént, hanem PBKDF2-HMAC-SHA256 alapú hash formájában tárolódnak.

Admin jogosultság esetén a rendszer lehetővé teszi a falbejegyzések törlését. Ez külön végponton érhető el, és szerveroldali ellenőrzés védi.

5.3 Adatbázis-műveletek és optimalizálás

A repository réteg fontos szerepe, hogy a felsőbb rétegek ne függjenek közvetlenül az adattárolás konkrét típusától. Emiatt ugyanaz a logika memóriaalapú és Firestore-alapú tárolón is futtatható. Ez megkönnyíti a tesztelést és a későbbi bővítést.

A reakciók kezelése Firestore oldalon inkrementális frissítéssel történik, így a teljes dokumentum felülírása helyett csak a szükséges mezők módosulnak.

6 Kommunikáció és Integráció

6.1 Protokollok

A kommunikáció elsődleges protokollja a **HTTP/HTTPS**. A mobil kliens és a FastAPI backend JSON alapú REST hívásokat használ. A jelenlegi verzió nem használ WebSocketet vagy gRPC-t, mert az MVP funkcióihoz a kérés-válasz modell elegendőnek bizonyult.

6.2 Adatcsere formátumok

Az API minden fontos végpontja **JSON** formátumot használ. Ez egyszerűen kezelhető Flutterben és FastAPI oldalon is. Az alábbi példa a generálási végpont kérés szerkezetét mutatja:

Listing 2: Példa kérés a generálási végponthoz

```
1 {
2   "truth": "I'm late because I overslept.",
3   "style": "goofy"
4 }
```

6.3 Összehangolás

A mobilalkalmazás először hitelesíti a felhasználót, majd az azonosító token a védett hívásoknál `Authorization: Bearer ...` fejlécben továbbítja. Generáláskor a backend validálja a bemenetet, meghívja az AI szolgáltatást, elmenti az eredményt, majd a strukturált választ visszaküldi a kliensnek.

A nyilvános fal esetén a rendszer részben backend API-t, részben Firestore-alapú adatfolyamot használ, így a felhasználó közel valós idejű frissítést kaphat a megosztott tartalmakról.

7 Tesztelés és Eredmények

7.1 Egységtesztek és integrációs tesztek

A backend oldalon `pytest` és `httpx` segítségével készültek tesztek a fő üzleti folyamatokra:

- regisztráció és bejelentkezés;
- hibás hitelesítés kezelése;
- generálás mentése;
- timeout és upstream hiba kezelése;
- publikálás, kategóriák, toplista és admin törlés.

A mobil oldalon widget tesztek ellenőrzik például:

- hogy üres input esetén a `SAVE ME` gomb inaktív;
- hogy a generálás eredménye megjelenik;
- hogy a nyilvános fal elemei helyesen renderelődnek;
- hogy a drawer menüpontjai elérhetők.

7.2 Felhasználói tesztelés visszajelzései

A jelenlegi implementáció alapján a rendszer könnyen demonstrálható és intuitív. A legfontosabb pozitívum a gyors generálás, az egyértelmű kezdőképernyő és a humoros hangvételű prezentáció. Fejlesztendő területként azonosítható a részletesebb hibaüzenetek, a fejlettebb profilkezelés és az offline viselkedés javítása.

7.3 Teljesítménymérés

Formális terheléses mérés jelenleg még nem része a projektnek, viszont a rendszer architektúrája alkalmas erre. A backend válaszidejét döntően az AI upstream hívás befolyásolja. Helyi környezetben a saját API-réteg gyorsan reagál, a teljes végpont-latencia főként az OpenRouter válaszidejétől függ.

Ezért a teljesítmény optimalizálás jövőbeli irányai közé tartozhat:

- válasz-cache bevezetése gyakori mintákra;
- részletes request tracing;
- aszinkron naplózás és monitorozás;
- terheléses tesztek automatizálása.

8 Következtetések és Jövőbeli tervek

8.1 Elért eredmények összefoglalása

A projekt sikeresen megvalósított egy modern mobilalkalmazást, amely összekapcsolja a Flutter alapú frontendet, a FastAPI backend szolgáltatást, a Firestore adattárolást és egy AI-alapú generáló modult. Az MVP funkcionalitás működőképes, bemutatható és továbbfejleszthető alapot ad.

8.2 Ismert korlátok és hibák

A rendszer jelenlegi korlátai:

- az AI-válaszok minősége külső szolgáltatótól függ;
- a hitelesítés egy egyszerű, projektcélra megfelelő megoldás, nem teljes vállalati auth rendszer;
- a teljesítménymérés és CI/CD automatizálás még nem teljes körű;
- a dokumentált architektúra tovább finomítható külön vizuális diagramokkal.

8.3 Továbblépési lehetőségek

Egy 2.0-s verzióban a következő fejlesztések lennének indokoltak:

- fejlettebb profiloldal és felhasználói beállítások;
- részletes admin panel;
- push értesítések és napi kihívások;
- jobb analitika és használati riportok;
- erősebb production szintű telepítési és monitorozási folyamat.

9 Mellékletek és Referenciák

9.1 Irodalomjegyzék

Az irodalomjegyzék a `biblatex` csomag használatával kerül kezelésre. A dokumentum végén automatikusan megjelennek a felhasznált online dokumentációk és fejlesztői források.

9.2 Forráskód-részletek

Az alábbi példa a backend egyik fontos adatmodelljét mutatja, amely a generálási kérés validálásáért felel:

Listing 3: Pydantic modell a generálási kéréshez

```
1 class GenerateExcuseRequest(BaseModel):
2     truth: str = Field(min_length=1, max_length=240)
3     style: AlibiStyle
4
5     @field_validator('truth')
6     @classmethod
7     def validate_truth(cls, value: str) -> str:
8         cleaned = value.strip()
9         if not cleaned:
10             raise ValueError('Truth cannot be empty.')
11         return cleaned
```

Hivatkozások

- [1] Google Firebase. *Cloud Firestore Documentation*. 2026. URL: <https://firebase.google.com/docs/firestore> (elérés dátuma 2026. 03. 23.).
- [2] FastAPI. *FastAPI Documentation*. 2026. URL: <https://fastapi.tiangolo.com/> (elérés dátuma 2026. 03. 23.).
- [3] Flutter Team. *Flutter Documentation*. 2026. URL: <https://docs.flutter.dev/> (elérés dátuma 2026. 03. 23.).
- [4] Render. *Render Documentation*. 2026. URL: <https://render.com/docs> (elérés dátuma 2026. 03. 23.).